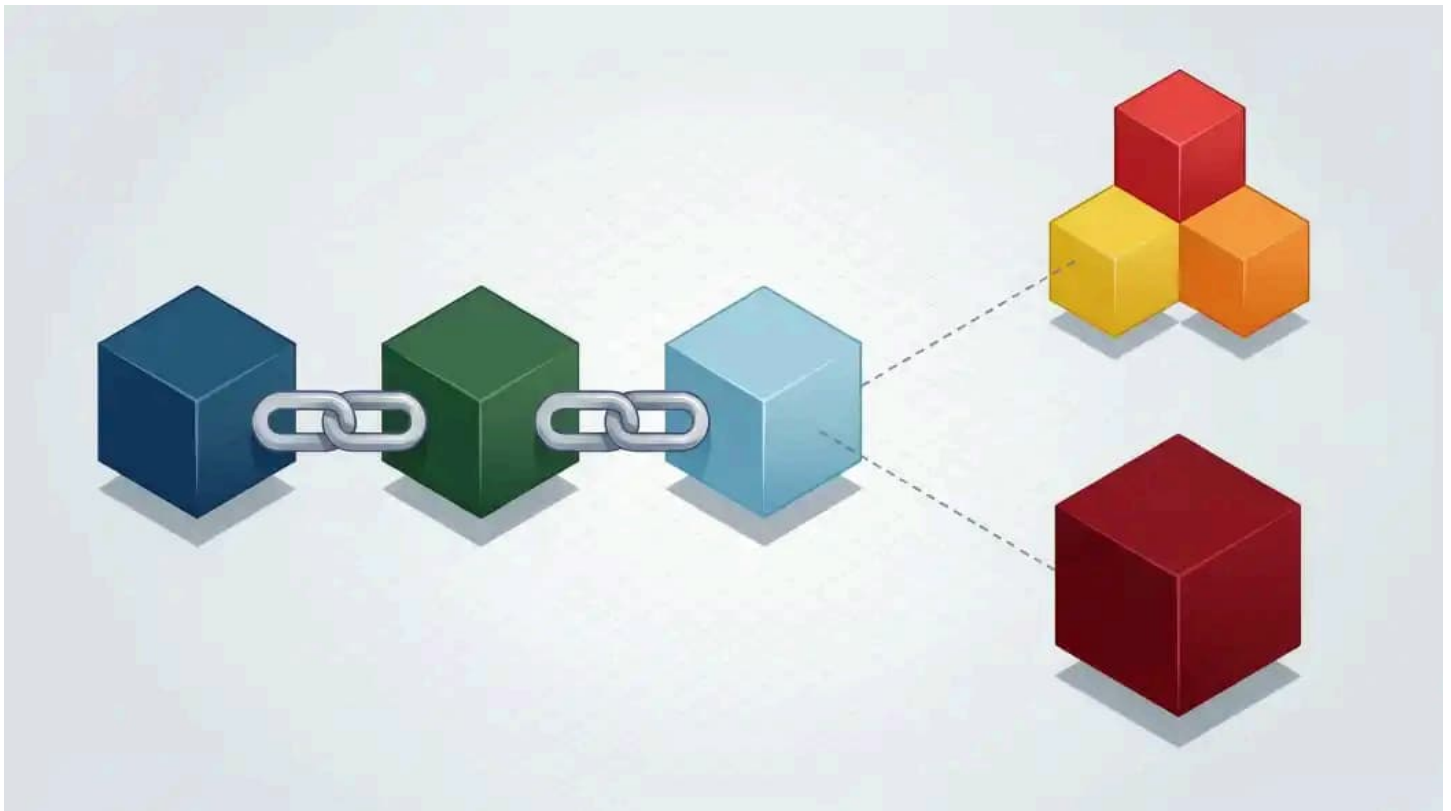


Speculative Decoding: Wie Multi-Token Prediction LLMs beschleunigt

18.06.2026 07:00 Uhr Dr. Christian Winkler



(Bild: Vanessa Bahr / KI / iX)

Spekulative Decoding-Verfahren beschleunigen die Tokenvorhersage. Multi-Token Prediction nutzt schnelle kleine Modelle und trennt Vorhersage von Verifikation.

Wer sich mit großen Sprachmodellen (LLMs) beschäftigt, weiß, dass das Generieren von Token Zeit und in der Cloud Geld kostet. Obwohl der Betrag für ein einzelnes Token gering ist, summieren sich die Kosten bei der Nutzung von LLMs als Coding-Assistenten oder agentische Anwendungen. Bei einer lokalen Installation muss man lange auf das finale Ergebnis warten; in der Cloud geht die Berechnung zwar schnell, wird aber teuer. Grund dafür ist der aufwendige Vorhersageprozess, bei dem Embeddings durch ein komplexes neuronales Netz propagieren, um ein einzelnes nächstes Token vorherzusagen. Ist das Token bestimmt, beginnt das Spiel von Neuem – so sagt das LLM ein Token nach dem anderen vorher. In fast allen Fällen ist dieser Prozess durch die Speicherbandbreite der ausführenden Hardware limitiert.

Seit geraumer Zeit spukt daher das Verfahren Multi-Token Prediction (MTP) durch die Forschungsabteilungen. Die Idee: Wenn ein Modell pro Durchlauf mehrere Token vorhersagt, spart das Rechenzeit und beschleunigt die Sprachmodelle obendrein. Allerdings sind die Algorithmen grundsätzlich darauf ausgelegt, nur ein einzelnes Token vorherzusagen, dann geht es autoregressiv weiter. Dabei spielt es keine Rolle, ob sich das Modell beim nächsten Token absolut sicher ist oder sich zwischen mehreren ähnlichen Alternativen entscheiden muss – der Rechenaufwand bleibt gleich.

IX-TRACT

- Das Speculative-Decoding-Verfahren Multi-Token Prediction (MTP) ist mit dem Release von Gemma 4 in den Mainstream gerückt.
 - MTP soll Rechenzeit beim Ausführen großer Sprachmodelle sparen, indem das Modell pro Durchlauf mehrere Token auf einmal vorhersagt statt immer nur das nächste.
 - Dafür bedient man sich eines Kniffs: Ein kleines, schwächeres Modell (Drafter) sagt in kurzer Zeit mehrere Token voraus, die das größere Modell (Target) dann schneller prüfen kann, als es selbst Token generiert.
 - Verschiedene Ansätze von Speculative Decoding werden schon seit Jahren erforscht, viele sind bereits in Frameworks wie vLLM, llama.cpp und MLX integriert und lassen sich damit ausprobieren.
-

MEHR ZUM THEMA KÜNSTLICHE INTELLIGENZ (KI)

Effiziente KI: Neue Modelle könnten Token-Kosten und RAM-Preise drastisch senken

Faire KI in der Hautmedizin: Wenn der Rechner Krebs erfindet [1]

KI-Kosten reduzieren: Wie man mit Prompt-Caching messbar Token sparen kann

Speculative Decoding: Wie Multi-Token Prediction LLMs beschleunigt [2]

Fable 5 im Test: Das kann das teuerste Anthropic-Modell [3]

Arbeiten mit KI: Studien widerlegen Mythos vom Job-Killer und Fachkräfte-Ersatz [4]

Von wegen Job-Killer: Warum KI Arbeitnehmer mehr belastet, statt sie zu ersetzen [5]

Warum Europa bei der KI-Entwicklung den Anschluss verliert [6]

Wofür Software-Entwickler KI-Tools nutzen – und wofür nicht [7]

Mit dem Release von Gemma 4 ist MTP in den Mainstream gerückt, da **Google einen Artikel zur Integration von MTP in Gemma veröffentlicht hat [8]**. Der Ansatz stammt aber schon aus dem Jahr 2022 [9]. Die Open-Source-Community hat mit **EAGLE [10]** und DFlash ähnliche Ansätze vorgestellt. Die daraus gewonnenen Erkenntnisse führen zu einer schnellen Verfügbarkeit in offenen Tools.



Prof. Dr. Christian Winkler beschäftigt sich speziell mit der automatisierten Analyse natürlichsprachiger Texte (NLP). Als Professor an der TH Nürnberg konzentriert er sich bei seiner Forschung auf die Optimierung der User Experience.

So funktioniert Multi-Token Prediction

Doch wie bringt man einem Modell bei, mehrere Token auf einmal vorherzusagen? Die überraschende Antwort lautet: Gar nicht. Stattdessen bedient man sich eines Kniffs und integriert ein kleineres Modell, das einzelne Token schneller vorhersagt als das große Modell. Das kleinere Modell heißt in diesem Zusammenhang Drafter, das Hauptmodell Target. Nachdem der Drafter einige Token einzeln generiert hat, überprüft das Target-Modell das Ergebnis am Stück und sagt selbst ein weiteres Token vorher. Im Überprüfen ist das Modell schneller als im Tokengenerieren. Da die vom Drafter generierten, noch nicht vom Target überprüften Token spekulativ sind, nennt sich das Verfahren Speculative Decoding.

Speculative Decoding spart bisher nur Zeit bei den Antworten der Modelle und nicht beim Prompt-Parsing, also der Verarbeitung der Nutzereingaben. Das ist besonders bei langen Prompts relevant, etwa wenn man sich ein Dokument zusammenfassen lassen möchte oder das Modell ein Programm analysieren soll. Beim Prompt-Parsing ist nach wie vor nur das große Modell aktiv. Zwar gibt es einige Ansätze, die auch dieses Parsing beschleunigen möchten, die Entwicklung steckt hier aber noch in den Kinderschuhen.

Die Abbildung verdeutlicht den Ablauf der Multi-Token Prediction. Zunächst sagt das Draft-Modell in drei Durchgängen drei Token vorher. Diese bestätigt das Target-Modell, während es das folgende Token „ist“ vorhersagt – denn beim Generieren des nächsten Tokens überprüft das Modell automatisch die vorherigen. Damit hat das Target-Modell vier Token in einem Durchlauf vorhergesagt. Mit diesen vier Vorhersagen als weiterem Kontext kann das Draft-Modell nun weitere drei Vorhersagen durchführen. Beim dritten Token kommt es zum Konflikt, das Target-Modell wählt ein eigenes vorhergesagtes Token. Dadurch hat das große Modell in diesem Durchlauf immer noch drei Token auf einmal generiert.

Prompt: Erkläre den Heise Verlag

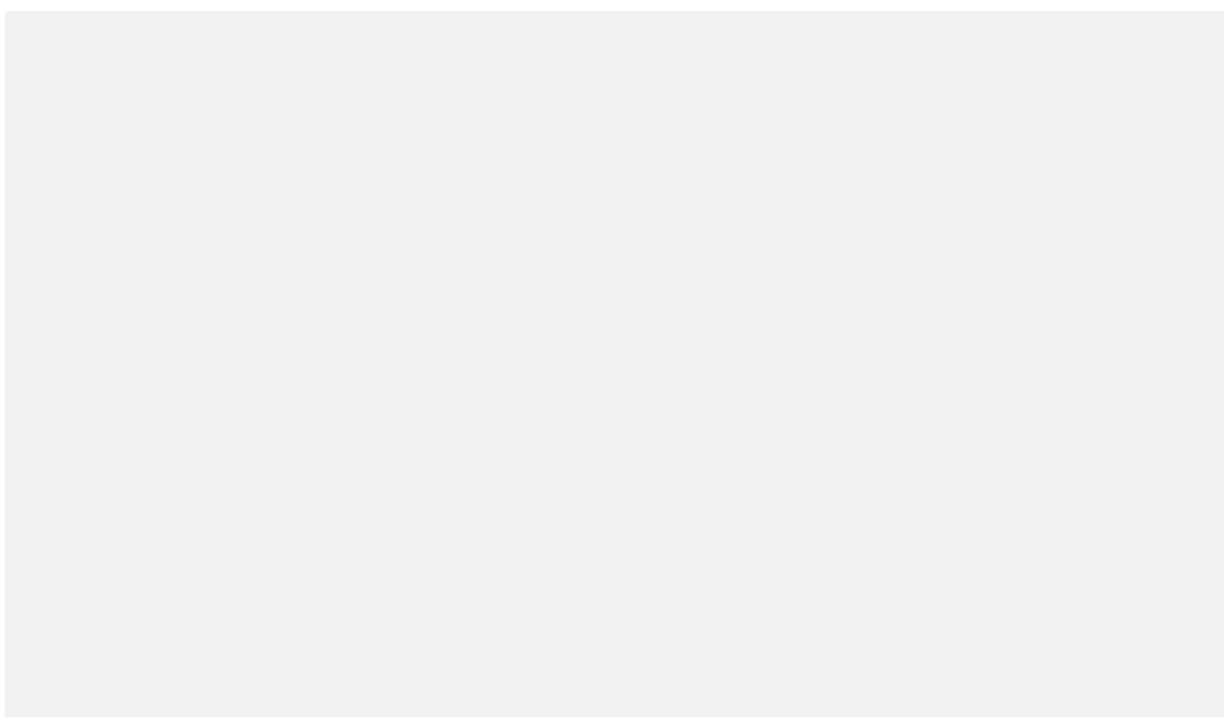


Bei der Multi-Token Prediction sagt ein kleineres Sprachmodell (Drafter) mehrere Token voraus, die das größere Hauptmodell (Target) dann überprüft. Im Konfliktfall generiert das Target-Modell ein eigenes Token.

In diesem Beispiel ergibt sich eine Akzeptanzrate von 5/6, also etwa 83 Prozent. Durch lediglich zwei Vorhersagen des Target-Modells stehen bereits sieben verifizierte Token zur Verfügung. Vernachlässigt man die Rechenzeit des Draft-Modells und geht von einer Geschwindigkeit von 10 Token/s für das Target-Modell aus, sagt es im Beispiel in 0,2 Sekunden sieben Token vorher, also effektiv 35 Token/s – ein Geschwindigkeitsgewinn ohne Verlust an Präzision. Das Verfahren entkoppelt die Tokenvorhersage von der Tokenverifikation.

Dabei hängt viel von der Qualität des Draft-Modells ab: Je genauer es die Token vorhersagt, desto häufiger akzeptiert das große Modell die Vorhersage und profitiert von der Arbeitsteilung. Besonders gut funktioniert der Prozess bei Token mit hoher Wahrscheinlichkeit. Die kann auch das schwächere Modell richtig vorhersagen, weil sie sich in den meisten Fällen als nächstes Token anbieten. So kann das kleine Modell beim deutschen Ausspruch „Vom Regen in ...“ mit sehr hoher Wahrscheinlichkeit „die“ und „Traufe“ richtig vorhersagen. Bei anderen Sätzen und komplexeren Themen ist das schwieriger. Die Akzeptanzrate hängt also von der Art der Daten ab, besonders gut funktioniert das mit Code in Programmiersprachen, weil der oft festen Regeln folgt.

Das Vorhersageverfahren funktioniert mit allen Sprachmodellen, wenn man für ein großes Modell einen entsprechenden Drafter findet. Dafür gibt es mehrere Herangehensweisen, an denen die Anbieter im Moment arbeiten.



[11]

Integrierte MTP-Modelle

Google favorisiert ein integriertes MTP-Modell und **beschreibt das Vorgehen sehr genau ausgerechnet in einem Artikel auf X [12]**. Das kleinere Modell muss speziell als Drafter-Modell trainiert sein, kann dafür aber auf interne Daten und Zustände des großen Modells zugreifen. Dazu gehört der Key-Value-Cache, der bei integrierten MTP-Modellen nur einmal benötigt wird. Gleiches gilt auch für versteckte Zwischenzustände und Aktivierungen, die das Verfahren zwischen den Modellen teilt. Google hat noch mehr Tricks auf Lager und verwendet für die kleinen E2B- und E4B-Modelle Clustering-Techniken im Embedder, um noch mehr Zeit zu sparen.

Je nach Modellart gibt es für das Verfahren mehrere Hürden. So ist etwa bei MoE-Modellen mit einer kleinen Batchgröße das Routing nicht so einfach – sagt ein Experte im kleinen Modell ein Token vorher, könnte ein anderer Experte des großen Modells zu einem anderen Ergebnis kommen.

Dennoch sieht Google auch in solchen Fällen eine architekturabhängige Beschleunigung von mehr als Faktor zwei. Andere Anbieter wie Qwen haben ähnliche integrierte Modelle. Der Vorteil dabei ist, dass sich das in Tools wie llama.cpp integrieren lässt.

Block-Diffusion-Modell (DFlash)

Im Februar haben Forscher der University of California San Diego mit DFlash einen weiteren Ansatz für die **parallele Vorhersage mehrerer Token auf arXiv veröffentlicht [13]**. DFlash nutzt dafür ein Diffusions-LLM. Diffusionsmodelle kommen bisher vor allem bei der Bildgenerierung zum Einsatz und sagen aufgrund ihrer Architektur immer mehrere Token gleichzeitig voraus. Im Einsatz als Draft-Modell sagt ein Diffusions-LLM tatsächlich mehrere Token auf einmal vorher.

Da autoregressive Modelle bei der Tokenvorhersage für Texte bisher besser funktionieren als Diffusions-LLMs, nutzen die Autoren die Diffusionsvariante nur für das Draft-Modell, während ein

großes autoregressives Modell die vorgeschlagenen Token verifiziert, akzeptiert oder verwirft. Der Vorteil des spekulativen Diffusionsmodells ist dessen Geschwindigkeit sowie die Möglichkeit, es speziell auf die Architektur und die Vorhersagen des großen Modells anzupassen.

Das Verfahren hat aber auch ein paar Nachteile: So muss die Software zur Vorhersage mit Diffusionsmodellen umgehen können und man benötigt für jedes große Modell ein eigenes, speziell trainiertes Diffusionsmodell. Trainingsdaten dafür stehen in Massen bereit, weil das große Modell diese erzeugen kann. Die Trainingsdaten lassen sich dabei auch für spezielle Domänen erzeugen – die Vorhersagen sollten hier dann zuverlässiger sein. Der Geschwindigkeitsgewinn fällt entsprechend hoch aus.

Ausprobieren mit vLLM

Wie häufig bei neuen Ansätzen gibt es eine Fülle von Software, die sie integriert. Wir betrachten unterschiedliche Varianten, der Einsatz wurde mit einer Nvidia RTX 4090 getestet. Dabei haben wir den Modellen zwei Prompts gegeben: für einen deutschen Text „Erkläre den Heise Verlag“, für eine einfache Programmieraufgabe „Write a quicksort in Python“. Für eine statistisch valide Aussage liefen die Modelle zehnmal pro Anfrage. Die folgenden Ergebnisse sind der Mittelwert der zehn Versuche.

vLLM unterstützt MTP in der integrierten Form für verschiedene Modelle [14], etwa für die Gemma-4-Modelle von Google oder für Qwen3.6. Dabei kann man konfigurieren, wie viele Token das Draft-Modell vorhersagen soll, im Test sind ein und drei Token (MTP1 und MTP3). Die Trefferwahrscheinlichkeit hängt vom Inhalt ab, bei Programmcode funktionierte das Experiment besser als bei einem Text in deutscher Sprache, weswegen sich die Performance unterscheidet. Eine Messung mit Qwen3.6-27B war leider auf der RTX 4090 nicht möglich, da das Modell zu groß für den Speicher der GPU ist, daher wurde dafür eine RTX 6000 verwendet.

Das folgende Listing zeigt den Aufruf, die nächste Tabelle die Ergebnisse mit vLLM. Eine wie von Google gemessene Verdopplung der Geschwindigkeit gelang nur bei einfachen Prompts, die Programme generieren. Dort ist auch die Annahmewahrscheinlichkeit der Token sehr hoch, da die kleinen Modelle gut im Vervollständigen von Code sind.

Listing 1: MTP mit Gemma 4 und Qwen3.6 bei vLLM

```
vllm serve google/gemma-4-E2B-it
--tensor-parallel-size 1 --max-model-len 8192
--speculative-config '{"method":"mtp",
                      "model":"gg-hf-am/gemma-4-E2B-it-assistant",
                      "num_speculative_tokens":1/3}'
```

Alternativer Aufruf mit Quantisierung:

```
vllm serve cyankiwi/gemma-4-E2B-it-AWQ-INT4
--tensor-parallel-size 1 --max-model-len 8192
--speculative-config '{"method":"mtp",
```

```
"model": "gg-hf-am/gemma-4-E2B-it-assistant",
"num_speculative_tokens": 1/3}
```

Tabelle: Ergebnisse in Token/s für vLLM (Version 0.21)

Modell	Prompt	vLLM	vLLM MTP1	vLLM MTP3	max. Speedup
Gemma 4 A4B	Heise	161	182	200	24 %
	Python	161	214	300	86 %
Gemma 4 A4B AWQ	Heise	272	290	300	10 %
	Python	272	335	470	73 %
Qwen3.6-27B AWQ	Heise	70	–	100	43 %
	Python	70	–	135	93 %

llama.cpp und BeeLlama.cpp

llama.cpp ist besonders für quantisierte Modelle bekannt geworden, die auch auf CPUs einigermaßen schnell laufen. Mittlerweile unterstützt llama.cpp jedoch auch verschiedene GPUs, die es mittels CUDA oder auch über die Vulkan-API ansprechen kann. Seit Mitte Mai 2026 ist MTP in den Main-Branch von llama.cpp integriert. Seitdem kann man MTP für Qwen3.6 mit llama.cpp nutzen, das funktioniert für die Modelle 27B und 35B-A3B. Das folgende Listing zeigt den Aufruf. Auch hier ergeben sich bei den unterschiedlichen Aufgaben Schwankungen bei der Performance.

Listing 2: Aufruf der Modelle bei llama.cpp

```
llama-server -m Qwen3.6-27B-UD-Q4_K_XL.gguf -np 1 --kv-unified
-ngl all -b 2048 -ub 256 --ctx-size 12280 --flash-attn on
--cache-ram 0 --jinja --no-mmap --mlock --no-host
--log-timestamps --log-prefix --log-colors off --reasoning on
--chat-template-kwarg '{"preserve_thinking":true}'
--temp 0.6 --top-k 20 --min-p 0.0
--host 0.0.0.0
--spec-default --spec-type draft-mtp --spec-draft-n-max 1/3/5
```

BeeLlama ist ein Fork von llama.cpp und nutzt ein DFlash-Modell zur spekulativen Vorhersage. Damit hat es sich von der llama.cpp-Entwicklung entkoppelt – MTP ist bei BeeLlama.cpp schon länger möglich. BeeLlama.cpp erreicht damit Beschleunigungsraten, die denen von vLLM und llama.cpp fast gleichkommen. Praktisch ist, dass man die Anzahl der vorhergesagten Token nicht manuell vorgeben muss – das erledigt BeeLlama.cpp dynamisch. Das folgende Listing zeigt den Aufruf bei BeeLlama, die nächste Tabelle den Vergleich mit llama.cpp.

Listing 3: Aufruf bei BeeLlama.cpp

```
llama-server -m Qwen3.6-27B-Q4_K_M.gguf
--spec-draft-model dflash-draft-3.6-q4_k_m.gguf --spec-type dflash
--spec-dflash-cross-ctx 1024
-np 1 --kv-unified -ngl all --spec-draft-ngl all
-b 2048 -ub 256 --ctx-size 122800
--cache-type-k turbo4 --cache-type-v turbo3_tcq
--flash-attn on --cache-ram 0 --jinja --no-mmap --mlock --no-host
--metrics --log-timestamps --log-prefix --log-colors off
--reasoning on --chat-template-kwarg '{"preserve_thinking":true}'
--temp 0.6 --top-k 20 --min-p 0.0 --host 0.0.0.0
```

Tabelle: Token/s für llama.cpp und BeeLlama.cpp

Modell	Prompt	llama.cpp	llama.cpp MTP1	llama.cpp MTP3	llama.cpp MTP5	BeeLlama.cpp	max. Speed
Qwen3.6 GGUF	Heise	46	67	76	67	66	65 %
	Python	46	89	110	89	115	150 %

llama.cpp-Version: b9305-1-g5d246a792; BeeLlama.cpp-Version: main-b9459-07ac3ce

MLX auf Apple-Geräten

Auch mit Apples MLX-Framework lassen sich mittlerweile MTP-Modelle verwenden. Als Software bietet sich oMLX an, damit gelang im Test leider keine Beschleunigung des Qwen3-27B-Modells in der Quantisierungsstufe vier Bit. Das könnte am verwendeten M2-Ultra-Prozessor liegen, für den die Software noch nicht optimiert ist.

Etwas bessere Ergebnisse kann man mit MTPLX [15] erzielen, wenn man die dort vorgeschlagenen optimierten Modelle verwendet. dflash-mlx [16] bringt das Block-Diffusion-Modell DFlash unter macOS zum Laufen, auch hier wieder nur mit mäßiger Beschleunigung. Glaubt man den Resultaten auf den jeweiligen Websites, so sieht das Resultat bei neuerer Hardware deutlich besser aus. Bei älterer Hardware scheinen die Beschleunigungen noch optimierbar.

Tabelle: Token/s für MLX (0.31.3), MTPLX (0.3.7), oMLX (0.3.12) und dflash-mlx (0.1.7)

Modell	Prompt	MLX	MTPLX	oMLX	dflash-mlx	max. Speedup
Qwen3-27B mlx	Heise	33	31	30	27	0 %
	Python	33	40	30	39	21 %

Gamechanger MTP

Die aktuellen Entwicklungen der verschiedenen Varianten von Multi-Token Prediction sind spannend zu beobachten. Auch wenn die Installation und das Handling noch aufwendig sind, zahlt sich der Geschwindigkeitsgewinn schnell aus. Besonders aus älterer Hardware lässt sich damit noch Leistung herauskitzeln: Mit einer (älteren) RTX 4090 und dem 27B-Modell von Qwen3.6 knapp 100 Token pro Sekunde zu generieren, fühlt sich fast genauso flüssig an wie Claude und ChatGPT.

Weil das Qwen3.6-27B-Modell so stark ist, eignet es sich auch für agentische Aufgaben oder als Coding-Assistent. Dabei bleibt alles auf dem lokalen Computer. MTP könnte der entscheidende Performance-Boost sein, lokale LLMs zu nutzen. Das geschieht ohne Verluste in der Vorhersagequalität. Es wird interessant sein zu beobachten, ob sich auch beim Prompt-Parsing ähnliche Optimierungen erreichen lassen.

Multi-Token Prediction könnte sich auf die Wirtschaftlichkeit großer Sprachmodelle auswirken. Durch die Geschwindigkeitsgewinne könnte sich die Last in bestehenden Rechenzentren besser verteilen. Ob durch integrierte MTP-Modelle oder spezialisierte Drafter-LLMs: Die Entwicklung in diesem Bereich nimmt gerade an Fahrt auf.

data2day 2026: Die Konferenz für Data Scientists, Data Engineers und Data Teams

Vom 7. bis 8. Oktober 2026 versammelt die **data2day in Köln (Anmeldung und weitere Infos)** [17] sowohl Einsteiger:innen in Data Science und Machine Learning als auch Fortgeschrittene, die sich mit Architekturen, Prozessen und Vorgehensmodellen befassen. Agentic AI, LLMs und Evaluation ziehen sich wie ein roter Faden durch das zweitägige Vortragsprogramm. Schwerpunkte setzt die Konferenz zudem bei Datenarchitektur, Datenqualität und Datenprodukten. Industrielle und produktnahe Erfahrungsberichte sowie rechtliche Einordnungen zu Themen wie dem rechtssicheren Einsatz von KI in Unternehmen runden das Programm ab.





Interessierte können sich bis zum 13. August für die data2day zum Frühbucherpreis von 1.099 Euro (zuzüglich 19 Prozent Mehrwertsteuer) anmelden. Gruppen ab drei Personen erhalten im Ticketshop gestaffelte Rabatte. Hochschulangehörige, Studierende, Schülerinnen und Schüler erhalten auf Anfrage vergünstigte Tickets. (pst [18])

URL dieses Artikels:

<https://www.heise.de/-11332677>

Links in diesem Artikel:

- [1] <https://www.heise.de/hintergrund/Faire-KI-in-der-Hautmedizin-Wenn-der-Rechner-Krebs-erfindet-11327176.html>
- [2] <https://www.heise.de/hintergrund/Speculative-Decoding-Wie-Multi-Token-Prediction-LLMs-beschleunigt-11332677.html>
- [3] <https://www.heise.de/tests/Fable-5-im-Test-Das-kann-das-teuerste-Anthropic-Modell-11329086.html>
- [4] <https://www.heise.de/ratgeber/KI-im-Job-Studien-widerlegen-Mythos-vom-Job-Killer-11280789.html>

- [5] <https://www.heise.de/ratgeber/KI-im-Job-Warum-KI-Arbeit-verdichtet-und-Fachkraefte-kaum-entlastet-11280801.html>
- [6] <https://www.heise.de/hintergrund/Warum-Europa-bei-der-KI-Entwicklung-den-Anschluss-verliert-11313310.html>
- [7] <https://www.heise.de/ratgeber/Wofuer-Software-Entwickler-KI-Tools-nutzen-und-wofuer-nicht-11273894.html>
- [8] <https://blog.google/innovation-and-ai/technology/developers-tools/multi-token-prediction-gemma-4/>
- [9] <https://arxiv.org/abs/2211.17192>
- [10] <https://github.com/SafeAILab/EAGLE>
- [11] <https://www.heise.de/ix>
- [12] <https://x.com/googlegemma/status/2051694045869879749>
- [13] <https://arxiv.org/abs/2602.06036>
- [14] https://docs.vllm.ai/en/latest/features/speculative_decoding/mtp/
- [15] <https://github.com/youssofal/MTPLX>
- [16] <https://github.com/bstnxbt/dflash-mlx>
- [17] <https://heise.de/s/Nd5Kr>
- [18] <mailto:pst@ix.de>

Copyright © 2026 Heise Medien